

CHEAT SHEETS

A Complete Reference for Structured Documents & Data

[Document Structure](#)[Elements & Attributes](#)[CDATA & Entities](#)[Namespaces](#)[DTD](#)[XML Schema](#)[XPath](#)[XSLT](#)[DOM & SAX](#)[SOAP · RSS · SVG](#)

10 REFERENCE SHEETS

CONTENTS

FUNDAMENTALS

- 01 Basics & Document Structure
- 02 Elements & Attributes
- 03 Text, Data & Special Characters
- 04 Namespaces

VALIDATION & QUERYING

- 05 DTD — Document Type Definition
- 06 XML Schema (XSD)
- 07 XPath

PROCESSING & PRACTICE

- 08 XSLT
- 09 Parsing XML
- 10 Practical XML

Sheet 1

Basics & Document Structure

prolog · declaration · root element · well-formedness · parsers · file types

Sheet 2

Elements & Attributes

element syntax · nesting · empty elements · attributes · element vs attribute

Sheet 3

Text, Data & Special Characters

CDATA · entities · character refs · whitespace · encoding · Unicode

Sheet 4

Namespaces

xmlns · prefixes · default namespace · scope · XHTML · SOAP · SVG

Sheet 5

DTD — Document Type Definition

!ELEMENT · !ATTLIST · !ENTITY · content models · internal · external

Sheet 6

XML Schema (XSD)

schema structure · simple types · complex types · restrictions · xs:sequence

Sheet 7

XPath

axes · node selection · predicates · functions · absolute · relative paths

Sheet 8

XSLT

templates · apply-templates · for-each · if · choose · output methods

Sheet 9

Parsing XML

DOM · SAX · StAX · Python · JS · tree navigation · error handling

Sheet 10

Practical XML

SOAP · REST · RSS · SVG · config files · Maven · Android · XML to JSON

WHAT IS XML?

XML (eXtensible Markup Language) is a text-based format for storing and transporting structured data. Unlike HTML, XML has no predefined tags — you define your own tag names to describe your data. Every XML document must be **well-formed**: it must have exactly one root element, all tags must be properly nested and closed, and attribute values must be quoted. XML is case-sensitive: `<Book>` and `<book>` are different elements.

KEY FACTS

- › Defined by the W3C in 1998; current spec is XML 1.0 (5th ed.)
- › File extensions: `.xml`, `.xsl`, `.xsd`, `.svg`, `.rss`
- › Default encoding is UTF-8; declare others in the prolog
- › Whitespace outside elements is preserved by default
- › Comments use `<!-- text -->` syntax; cannot nest
- › Processing instructions use `<?target data?>`
- › Common parsers: browser DOM, Python `xml.etree`, Java JAXP

ANATOMY OF AN XML DOCUMENT

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- XML prolog ↑ — always line 1 if present -->

<!-- Optional DOCTYPE declaration -->
<!DOCTYPE catalog SYSTEM "catalog.dtd">

<catalog>                                <!-- root element (exactly one) -->

  <book id="bk101" lang="en">             <!-- element with attributes -->
    <title>XML Developer's Guide</title>
    <author>Gambardella, Matthew</author>
    <price currency="USD">44.95</price>
  </book>

  <book id="bk102" lang="en">
    <title>Midnight Rain</title>
    <author>Ralls, Kim</author>
    <price currency="USD">5.95</price>
    <note />                             <!-- empty element (self-closing) -->
  </book>

</catalog>
```

NOTES

- `<?xml?>` — prolog; optional but strongly recommended; must be first line with no whitespace before it
- `version="1.0"` — only version in wide use; XML 1.1 exists but is rarely needed
- `encoding="UTF-8"` — omitting this defaults to UTF-8; declare explicitly for clarity
- `<catalog>` — every document must have exactly one root element wrapping all content
- Attributes must always be quoted — single or double quotes both valid
- `<note />` — self-closing form for empty elements; equivalent to `<note></note>`
- Tags are case-sensitive: opening and closing tags must match exactly

WELL-FORMEDNESS RULES

RULE	EXAMPLE — WRONG → RIGHT
One root element	<code><a/></code> → wrap in <code><root></code>
Tags must close	<code><p>text</code> → <code><p>text</p></code>
Proper nesting	<code><a></code> → <code><a></code>
Quoted attributes	<code>id=42</code> → <code>id="42"</code>
Case-sensitive tags	<code><Book>...</book></code> → match case
Escape special chars	<code><</code> in text → <code>&lt;</code> ;
Unique attr names	<code>id="a" id="b"</code> → only one id

COMMON XML FORMATS

FORMAT	USE CASE
SVG	Scalable vector graphics
RSS / Atom	News & podcast feeds
SOAP	Web service messages
XHTML	HTML rewritten as XML
pom.xml	Maven project config
AndroidManifest	Android app config
Office Open XML	.docx, .xlsx internals
KML	Geographic data (Google Maps)

COMMON MISTAKES

- › Whitespace or a BOM before the `<?xml?>` prolog — parsers will reject the document
- › Using bare `<` or `&` in text content — must escape as `<` / `&`;
- › Mismatched tag case: `<Title>...</title>` is not valid XML

TIPS

- › Always declare `encoding="UTF-8"` in the prolog, even though it's the default — it prevents encoding surprises when files are moved between systems
- › Validate with an online XML linter or your editor before parsing programmatically
- › Use attributes for metadata (IDs, types) and child elements for content that may grow

SUMMARY

- › `<?xml version="1.0" encoding="UTF-8"?>` — prolog goes on line 1
- › One root element wraps all content — no siblings at the top level
- › Well-formed = closed tags, proper nesting, quoted attributes, escaped specials
- › XML is case-sensitive; opening and closing tags must match exactly

ELEMENTS AND ATTRIBUTES

An **element** is the primary building block of XML, consisting of a start tag, content, and an end tag: `<price>44.95</price>`. Elements can contain text, other elements, or both (mixed content). An **attribute** provides additional metadata directly on the start tag in `name="value"` form. The key design question is always: should this be an element or an attribute? Use elements for data that might expand, repeat, or need child structure; use attributes for concise, single-value metadata like IDs and types.

ELEMENT NAMING RULES

- Names must start with a letter or underscore (`_`)
- Can contain letters, digits, hyphens, underscores, periods
- Cannot start with a digit or hyphen
- Cannot start with `xml` (any case) — reserved
- No spaces in names
- Case-sensitive: `<Book>` $\neq$ `<book>`
- Convention: use camelCase or kebab-case consistently

ELEMENT & ATTRIBUTE SYNTAX

```
<!-- Basic element: start tag · content · end tag -->
<title>XML Developer's Guide</title>

<!-- Empty element – two equivalent forms -->
<br />
<br></br>

<!-- Element with attributes -->
<book id="bk101" lang="en" available="true">
  <title>XML Developer's Guide</title>
  <price currency="USD">44.95</price>
</book>

<!-- Mixed content (text + child elements) -->
<description>
  A <em>comprehensive</em> guide to XML.
</description>

<!-- Multiple same-name children (repetition is fine) -->
<catalog>
  <tag>fiction</tag>
  <tag>reference</tag>
  <tag>xml</tag>
</catalog>
```

NOTES

- `<br />` — self-closing syntax; the space before `/` is optional but improves compatibility with older parsers
- `id="bk101"` — attribute order on a tag is not significant; parsers may reorder them
- Attribute values must always be quoted — single or double quotes are both valid, but pick one style and be consistent
- An element cannot have two attributes with the same name
- Mixed content (text and child elements together) is valid but makes XPath and schema validation more complex — avoid it when possible
- Repeated child elements like `<tag>` are idiomatic XML for lists

ELEMENT VS. ATTRIBUTE — WHEN TO USE

USE ELEMENT	USE ATTRIBUTE
Content that can repeat	Single, non-repeating value
Data with child structure	Simple scalar (ID, code, flag)
Human-readable content	Machine-readable metadata
Data you query with XPath	Classifiers and identifiers
Values that may grow/change	Values fixed by schema/type
<code><price>44.95</price></code>	<code>currency="USD"</code>

CONTENT MODELS

MODEL	DESCRIPTION
Element content	Contains only child elements, no direct text
Text content	Contains only text (character data)
Mixed content	Contains both text and child elements
Empty	No content; self-closing or open+close
Any	Any combination (used in flexible schemas)

COMMON MISTAKES

- Duplicate attribute names on the same element — `id="a" id="b"` is invalid; only one per name allowed
- Starting element names with a digit: `<1item>` is invalid; use `<item1>`
- Unquoted attribute values: `id=42` fails — must be `id="42"`

TIPS

- Always add an `id` or `key` attribute on repeating elements — makes XPath selection and merging far easier later
- Avoid mixed content models unless you're authoring document-style XML (like DocBook or DITA)
- Stick to one attribute quoting style across the whole document — double quotes are the convention

SUMMARY

- Elements hold data; attributes hold metadata on elements
- Empty elements can self-close: `<tag />`
- Attribute values must always be quoted; no duplicate names on one element
- Use elements for repeatable/expandable data, attributes for scalar identifiers

TEXT CONTENT & ESCAPING

Text inside XML elements is called **character data (CDATA)**. Five characters have special meaning to the XML parser and must be escaped when they appear in text content or attribute values: `<`, `>`, `&`, `'`, and `"`. For longer blocks of text containing many special characters — such as embedded code or HTML — wrap the block in a **CDATA section** (`<![CDATA[ ... ]]>`), which tells the parser to treat the entire block as raw text. Character references let you insert any Unicode code point directly using either decimal (`A`) or hex (`A`) notation.

ESCAPING & CDATA EXAMPLES

```
<!-- — Escaped special characters ————— -->
<formula>a &lt; b &amp;amp; b &gt; c</formula>
<!-- renders as:  a < b && b > c -->

<title lang=&apos;en&apos;>O&apos;Brien</title>

<!-- — CDATA section – no escaping needed inside ————— -->
<script>
  <![CDATA[
    if (a < b && c > d) {
      console.log("raw HTML & JS here");
    }
  ]]>
</script>

<!-- — Character references ————— -->
<symbol>&#169;</symbol>      <!-- © -->
<symbol>&#x00A9;</symbol>    <!-- © (hex) -->
<symbol>&#8364;</symbol>     <!-- € -->

<!-- — Whitespace in text content ————— -->
<desc xml:space="preserve">  indented    text  </desc>
```

PREDEFINED ENTITIES

- › `<` → `<` (less-than)
- › `>` → `>` (greater-than)
- › `&` → `&` (ampersand)
- › `'` → `'` (apostrophe)
- › `"` → `"` (double quote)
- › `A` → `A` (decimal char ref)
- › `A` → `A` (hex char ref)

NOTES

- `<![CDATA[]]>` — the sequence `[]]` cannot appear inside a CDATA block; split into two sections if needed
- `<` is mandatory in text; `>` is only strictly required after `[]]` but escaping it everywhere is best practice
- `'` and `"` — only strictly required inside attribute values using the matching quote character
- Decimal char refs (`©`) and hex refs (`©`) are equivalent; hex is more common for Unicode
- `xml:space="preserve"` — instructs processors to preserve all whitespace in this element; default behavior is application-defined
- Encoding declared in the prolog must match the actual file bytes — UTF-8 is safe for all Unicode

WHITESPACE HANDLING

CONTEXT	BEHAVIOR
Text content	Preserved exactly as written by default
Between elements	Technically preserved; often ignored by apps
<code>xml:space="preserve"</code>	Signals processor to keep all whitespace
<code>xml:space="default"</code>	Application may normalize whitespace
Attribute values	Leading/trailing spaces stripped by parser
CDATA sections	All whitespace preserved verbatim

COMMON ENCODINGS

ENCODING	NOTES
UTF-8	Default; handles all Unicode; recommended
UTF-16	Fixed 2-byte (BOM required); used in some Windows APIs
ISO-8859-1	Latin-1; legacy Western European documents
US-ASCII	7-bit only; subset of UTF-8
windows-1252	Windows legacy; superset of Latin-1

COMMON MISTAKES

- › Using bare `&` in text — e.g. `AT&T` must be `AT&T` or parser throws
- › Embedding `[]]>` inside a CDATA block — split the section at that point
- › Declaring `encoding="UTF-8"` but saving the file as UTF-16 — the mismatch causes parse failures

TIPS

- › Use CDATA sections for any content containing HTML, code snippets, or SQL — it eliminates the need to escape every special character
- › Prefer UTF-8 for all new XML documents — it is universally supported and the XML default
- › Add `xml:space="preserve"` to elements where indentation or line breaks carry meaning

SUMMARY

- › 5 predefined entities: `<`, `>`, `&`, `'`, `"`;
- › `<![CDATA[...]]>` — raw text block; no escaping needed inside
- › Char refs: `©` (decimal) or `©` (hex) for any Unicode point
- › UTF-8 is the default and recommended encoding for all XML

WHAT ARE XML NAMESPACES?

Namespaces prevent element name collisions when combining XML vocabularies from different sources — for example, merging an HTML `<table>` with a furniture `<table>`. A namespace is a URI that uniquely identifies a vocabulary. It is declared with the `xmlns` attribute: `xmlns="URI"` for a **default namespace** (applies to all unprefixes elements in scope) or `xmlns:prefix="URI"` for a **prefixed namespace**. The URI is just an identifier — it does not need to resolve to a real web page.

NAMESPACE RULES

- › A namespace URI is an identifier, not a URL — it need not be accessible
- › Namespace declarations are inherited by all descendant elements
- › A prefix can be overridden on a child element
- › Default namespace does *not* apply to attributes — attributes without a prefix have no namespace
- › The prefix `xml` is predefined and always maps to `http://www.w3.org/XML/1998/namespace`
- › The prefix `xmlns` itself is reserved and cannot be used as a regular prefix

NOTES

- `xmlns="URI"` — default namespace; all unprefixes child elements inherit this namespace automatically
- `xmlns:h="URI"` — binds prefix `h` to the URI; elements must be written as `<h:element>`
- Prefixes are arbitrary shorthand — `h`, `html`, and `xhtml` all work if bound to the same URI
- Two elements with the same local name but different namespace URIs are completely different elements
- `xmlns:soap="..."` in the SOAP example — declaring on the root is idiomatic; avoids repeating on every element
- To un-declare a default namespace on a child: `xmlns=""`

NAMESPACE SYNTAX

```
<!-- — Default namespace ————— -->
<html xmlns="http://www.w3.org/1999/xhtml">
  <head><title>Page</title></head>
</html>
<!-- all unprefixes elements belong to XHTML namespace -->

<!-- — Prefixed namespaces ————— -->
<root
  xmlns:h="http://www.w3.org/1999/xhtml"
  xmlns:f="http://example.com/furniture">

  <h:table>                <!-- HTML table -->
    <h:tr><h:td>Cell</h:td></h:tr>
  </h:table>

  <f:table>                <!-- furniture table -->
    <f:width>80cm</f:width>
    <f:length>120cm</f:length>
  </f:table>

</root>

<!-- — SOAP envelope (real-world example) ————— -->
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <soap:Body>
    <GetPrice xmlns="http://example.com/prices">
      <Item>Widget</Item>
    </GetPrice>
  </soap:Body>
</soap:Envelope>
```

COMMON NAMESPACE URIS

PREFIX	URI	USED IN
<code>xmlns</code>	<code>http://www.w3.org/1999/xhtml</code>	XHTML
<code>svg</code>	<code>http://www.w3.org/2000/svg</code>	SVG graphics
<code>xs</code> / <code>xsd</code>	<code>http://www.w3.org/2001/XMLSchema</code>	XML Schema
<code>xsi</code>	<code>http://www.w3.org/2001/XMLSchema-instance</code>	Schema instances
<code>soap</code>	<code>http://schemas.xmlsoap.org/soap/envelope/</code>	SOAP 1.1
<code>xsl</code>	<code>http://www.w3.org/1999/XSL/Transform</code>	XSLT stylesheets
<code>dc</code>	<code>http://purl.org/dc/elements/1.1/</code>	Dublin Core metadata
<code>rss</code>	<code>http://purl.org/rss/1.0/</code>	RSS 1.0 feeds

COMMON MISTAKES

- › Assuming the default namespace applies to attributes — it doesn't; attributes without a prefix have no namespace
- › Using the same prefix for different URIs in the same document — prefix-URI bindings must be consistent within a scope
- › Treating the namespace URI as a URL that must resolve — it's just a unique string identifier

TIPS

- › Declare all namespaces on the root element — it keeps them visible in one place and avoids repeated declarations
- › Use well-known prefixes (`xs`, `xsi`, `soap`) to match community convention and aid readability
- › When writing XPath against namespaced XML, you must register prefix-URI bindings in your XPath engine too

SUMMARY

- › `xmlns="URI"` — default namespace for all unprefixes elements in scope
- › `xmlns:prefix="URI"` — binds a prefix; elements use `prefix:name`
- › Namespace URIs are identifiers only — they don't need to resolve
- › Default namespace does not apply to attributes

WHAT IS A DTD?

A **Document Type Definition (DTD)** defines the legal structure of an XML document: which elements are allowed, in what order, how many times, and what attributes they carry. A DTD can be **internal** (embedded in the document's `<!DOCTYPE>` declaration) or **external** (a separate .dtd file referenced by URI).

DTDs use their own non-XML syntax with declarations starting with `<!`. While DTDs were the original XML validation mechanism, XML Schema (XSD) has largely replaced them in modern workflows because XSD supports data types and namespaces.

DTD LIMITATIONS

- › No data type support — all values are treated as text strings
- › No namespace awareness
- › Cannot constrain element content to a specific number pattern (e.g. exactly 3)
- › Non-XML syntax — harder to process with standard XML tools
- › External DTDs must be fetched at validation time (network cost)
- › XSD or Relax NG preferred for new projects

NOTES

- (book+) — content model; + means one or more; see occurrence table below
- #PCDATA — Parsed Character Data; element contains text (may include entity references)
- !ATTLIST — declares attributes for an element; can list multiple attributes in one declaration
- ID — attribute type meaning the value must be unique across the document
- #REQUIRED — attribute must be present; #IMPLIED = optional; #FIXED "val" = fixed value
- !ENTITY pub "... " — defines a reusable text entity; referenced as &pub; anywhere in the document
- SYSTEM vs PUBLIC — SYSTEM is a private URI; PUBLIC identifies a well-known DTD with a fallback URI

CONTENT MODEL SYMBOLS

SYMBOL	MEANING	EXAMPLE
(none)	Exactly once	(title)
?	Zero or one (optional)	(note?)
*	Zero or more	(tag*)
+	One or more	(book+)
,	Sequence (in order)	(a, b, c)
	Choice (either/or)	(a b)
EMPTY	No content allowed	EMPTY
ANY	Any content	ANY

COMMON MISTAKES

- › Declaring elements out of order in a sequence model — (a, b, c) enforces that exact order in the document
- › Using ID type but not ensuring uniqueness — parsers will error on duplicate ID values
- › Expecting DTD to validate data types like numbers or dates — it can't; use XSD for that

TIPS

- › Use **!ENTITY** to define repeated strings (company names, copyright text) — a single change updates the whole document
- › Prefer external DTDs for shared schemas; internal DTDs are fine for quick one-off validation
- › For new projects requiring validation, use XSD instead — it supports namespaces and data types that DTDs cannot express

DTD SYNTAX

```

<!-- — Internal DTD ————— -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE catalog [
    <!--ELEMENT catalog (book+)>
    <!--ELEMENT book (title, author, price, note?)>
    <!--ELEMENT title (#PCDATA)>
    <!--ELEMENT author (#PCDATA)>
    <!--ELEMENT price (#PCDATA)>
    <!--ELEMENT note (#PCDATA)>

    <!--ATTLIST book
        id ID #REQUIRED
        lang CDATA "en"
        available (yes|no) "yes"
    -->

    <!--ENTITY pub "Publisher Name Inc."-->
]>

<catalog>
    <book id="bk101" lang="en">
        <title>XML Guide</title>
        <author>&pub;</author>
        <price>44.95</price>
    </book>
</catalog>

<!-- — External DTD ————— -->
<!DOCTYPE catalog SYSTEM "catalog.dtd">
<!DOCTYPE html PUBLIC
    "-//W3C//DTD XHTML 1.0 Strict//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

```

ATTRIBUTE TYPES

TYPE	MEANING
CDATA	Any character string
ID	Unique identifier in document
IDREF	Reference to an ID value
IDREFS	Space-separated list of IDREFs
NMTOKEN	Valid XML name token
(a b c)	Enumeration of allowed values
NOTATION	Refers to a declared notation

SUMMARY

- › **!ELEMENT** — defines an element and its content model
- › **!ATTLIST** — declares attributes, their types, and defaults
- › **!ENTITY** — defines reusable text; referenced as **&name;**
- › Occurrence: **?** 0–1 · ***** 0–n · **+** 1–n · (none) exactly 1

WHAT IS XSD?

XML Schema Definition (XSD) is the W3C standard for describing the structure and data types of an XML document. Unlike DTDs, XSD schemas are themselves valid XML documents, support over 40 built-in data types (string, integer, date, boolean, etc.), and are fully namespace-aware. A schema is linked to an XML document via the `xsi:schemaLocation` attribute. XSD uses the namespace prefix `xs:` or `xsd:` bound to `http://www.w3.org/2001/XMLSchema`.

XSD VS DTD ADVANTAGES

- › 40+ built-in data types: `xs:integer`, `xs:date`, `xs:boolean`, etc.
- › Full namespace support
- › XSD is itself XML — parse and generate with standard XML tools
- › Supports default and fixed values, min/maxOccurs
- › Extensible: derive new types from existing ones
- › Supports `xs:any` for open-content models
- › Can import/include other schemas

NOTES

- `xs:sequence` — children must appear in the declared order; use `xs:all` for any order, `xs:choice` for one-of
- `minOccurs="0"` — makes element optional (default is 1); `maxOccurs="unbounded"` allows unlimited repetitions
- `xs:complexType` — element that contains children or attributes; `xs:simpleType` — element with text only
- `xs:restriction` — derives a new type by constraining an existing one; facets include `xs:minLength`, `xs:pattern`, `xs:enumeration`
- `use="required"` — attribute must be present; `use="optional"` (default) or `use="prohibited"`
- `type="xs:ID"` — value must be unique in the document, just like DTD's `ID` type

COMMON BUILT-IN TYPES

TYPE	DESCRIPTION
<code>xs:string</code>	Any text string
<code>xs:integer</code>	Whole number (no decimal)
<code>xs:decimal</code>	Arbitrary-precision decimal
<code>xs:boolean</code>	true or false
<code>xs:date</code>	YYYY-MM-DD format
<code>xs:dateTime</code>	YYYY-MM-DDThh:mm:ss
<code>xs:anyURI</code>	URI or URL value
<code>xs:ID</code>	Unique identifier in document
<code>xs:IDREF</code>	Reference to an <code>xs:ID</code> value

COMMON MISTAKES

- › Confusing `xs:sequence` (ordered) with `xs:all` (any order) — sequence is far more common and enforces child element ordering
- › Forgetting `minOccurs="0"` to make an element optional — default is required (`minOccurs=1`)
- › Linking the schema with the wrong namespace URI in `xsi:schemaLocation` — the namespace must exactly match the schema's `targetNamespace`

TIPS

- › Name complex types with a `Type` suffix (`BookType`) to distinguish type definitions from element declarations
- › Use `xs:pattern` with a regex to validate formats like postal codes, phone numbers, or version strings
- › Validate with `xmlint --schema schema.xsd doc.xml` on the command line, or use an IDE plugin for immediate feedback

XSD SYNTAX

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- — Simple element ————— -->
  <xs:element name="title" type="xs:string"/>
  <xs:element name="price" type="xs:decimal"/>
  <xs:element name="inStock" type="xs:boolean"/>

  <!-- — Complex type with sequence of children ————— -->
  <xs:complexType name="BookType">
    <xs:sequence>
      <xs:element name="title" type="xs:string"/>
      <xs:element name="author" type="xs:string" maxOccurs="unbounded"/>
      <xs:element name="price" type="xs:decimal"/>
      <xs:element name="note" type="xs:string" minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="id" type="xs:ID" use="required"/>
    <xs:attribute name="lang" type="xs:string" default="en"/>
  </xs:complexType>

  <!-- — Restriction: constrain a string ————— -->
  <xs:simpleType name="CurrencyCode">
    <xs:restriction base="xs:string">
      <xs:enumeration value="USD"/>
      <xs:enumeration value="EUR"/>
      <xs:enumeration value="GBP"/>
    </xs:restriction>
  </xs:simpleType>

  <!-- — Root element using our complex type ————— -->
  <xs:element name="catalog">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="book" type="BookType" maxOccurs="unbounded"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

RESTRICTION FACETS

FACET	CONSTRAINS
<code>xs:minLength</code>	Minimum string/list length
<code>xs:maxLength</code>	Maximum string/list length
<code>xs:pattern</code>	Regex pattern match
<code>xs:enumeration</code>	Allowed values list
<code>xs:minInclusive</code>	Minimum numeric value ($\geq$)
<code>xs:maxInclusive</code>	Maximum numeric value ($\leq$)
<code>xs:totalDigits</code>	Max significant digits
<code>xs:fractionDigits</code>	Max digits after decimal

SUMMARY

- › `xs:element` + `xs:complexType` — define structured elements with children
- › `xs:sequence` / `xs:choice` / `xs:all` — control child ordering
- › `minOccurs` / `maxOccurs` — control repetition (default 1 each)
- › `xs:restriction` — constrain a base type with facets like `xs:pattern`, `xs:enumeration`

XPATH — XML PATH LANGUAGE

XPath is the query language for selecting nodes from an XML document. An XPath expression describes a path through the document tree, similar to a filesystem path. Expressions are evaluated relative to a **context node**. A path starting with `/` is *absolute* (from the document root); one without is *relative* (from the current node). XPath is used natively inside XSLT, XML Schema, and XQuery, and is exposed in most programming languages through DOM APIs and dedicated libraries.

NODE TYPES

- › **Element** — a tagged element (`<book>`)
- › **Attribute** — an attribute of an element (`@id`)
- › **Text** — text content inside an element
- › **Comment** — `<!-- ... -->`
- › **Processing instruction** — `<?target data?>`
- › **Document root** — the virtual top node above the root element
- › **Namespace** — namespace node on an element

PATH EXPRESSIONS & PREDICATES	
EXPRESSION	SELECTS
<code>/catalog</code>	Root <code><catalog></code> element
<code>/catalog/book</code>	All <code><book></code> children of <code><catalog></code>
<code>//book</code>	All <code><book></code> elements anywhere in document
<code>book/title</code>	<code><title></code> children of <code><book></code> (relative)
<code>//@id</code>	All <code>id</code> attributes anywhere
<code>book/@lang</code>	<code>lang</code> attribute of each <code><book></code>
<code>/catalog/book[1]</code>	First <code><book></code> (1-indexed)
<code>/catalog/book[last()]</code>	Last <code><book></code>
<code>//book[@id]</code>	Books that have an <code>id</code> attribute
<code>//book[@id='bk101']</code>	Book where <code>id</code> equals <code>bk101</code>
<code>//book[price>10]</code>	Books where <code><price></code> content <code>> 10</code>
<code>/catalog/*</code>	All element children of <code><catalog></code>
<code>/catalog/node()</code>	All children (elements, text, comments)
<code>.</code>	Current (context) node
<code>..</code>	Parent of current node

USEFUL XPATH FUNCTIONS	
FUNCTION	RETURNS
<code>text()</code>	Text node children of context node
<code>node()</code>	Any node type
<code>count(node-set)</code>	Number of nodes in set
<code>last()</code>	Position of last node in context
<code>position()</code>	Position of current node (1-based)
<code>contains(str, sub)</code>	True if <code>str</code> contains <code>sub</code>
<code>starts-with(str, pre)</code>	True if <code>str</code> starts with <code>pre</code>
<code>normalize-space(str)</code>	Trims and collapses whitespace
<code>string(node)</code>	String value of node
<code>not(expr)</code>	Boolean negation
<code>name()</code>	Local name of context node
<code>local-name()</code>	Name without namespace prefix

XPATH AXES		
AXIS	ABBREVIATED	SELECTS
<code>child::</code>	<code>el</code>	Direct children (default axis)
<code>attribute::</code>	<code>@attr</code>	Attributes of context node
<code>descendant::</code>	<code>//el</code>	All descendants at any depth
<code>descendant-or-self::</code>	<code>//</code>	Context node + all descendants
<code>parent::</code>	<code>..</code>	Direct parent node
<code>ancestor::</code>	<code>—</code>	All ancestors up to root
<code>ancestor-or-self::</code>	<code>—</code>	Context node + all ancestors
<code>following-sibling::</code>	<code>—</code>	Siblings after context node
<code>preceding-sibling::</code>	<code>—</code>	Siblings before context node
<code>self::</code>	<code>.</code>	The context node itself

COMMON MISTAKES

- › Forgetting XPath positions are 1-based: `[1]` is the first node, not `[0]`
- › Using `//` at the start of every expression — it scans the entire tree each time and is very slow on large documents; prefer a more specific path
- › XPath on namespaced XML without registering prefixes — the engine needs prefix-URI bindings to match namespaced elements

TIPS

- › Test XPath expressions interactively in browser DevTools: `$x("//book[@id='bk101']")` in the console
- › Use `normalize-space()` around text comparisons to ignore leading/trailing whitespace differences
- › Combine predicates: `//book[@lang='en'][price > 10]` — multiple predicates are ANDed together

SUMMARY

- › `/` — absolute path from root; no `/` = relative from context node
- › `//` — anywhere in document (descendant-or-self)
- › `@attr` — select attribute; `[predicate]` — filter a node-set
- › Positions are 1-based; `last()` returns final position in context

WHAT IS XSLT?

XSLT (XSL Transformations) is an XML-based language for transforming XML documents into other formats — HTML, plain text, another XML vocabulary, or any text output. An XSLT stylesheet is itself a well-formed XML document using the `xsl:` namespace. Processing works by matching nodes in the source tree with **templates** and outputting their transformed result. XSLT uses **XPath** expressions to select and test nodes. The XSLT processor applies templates recursively until the entire source tree has been processed.

CORE XSLT ELEMENTS

- › `xsl:stylesheet` — root element of every XSLT stylesheet
- › `xsl:template` — defines a rule matching a set of nodes
- › `xsl:apply-templates` — processes child nodes with matching templates
- › `xsl:value-of` — outputs the string value of an XPath expression
- › `xsl:for-each` — iterates over a node-set
- › `xsl:if` — conditional output
- › `xsl:choose` / `xsl:when` / `xsl:otherwise` — switch-style conditionals
- › `xsl:output` — declares output method and encoding

NOTES

- `match="/"` — matches the document root node (virtual node above the root element); this template fires first
- `xsl:apply-templates select="..."` — hands off processing to whichever template matches each selected node; without `select`, processes all children
- `xsl:value-of select="title"` — outputs the string value of `<title>`; equivalent to `XPath string(title)`
- `test="price > 10"` — XPath expression in `test`; use `>` because `>` would break XML parsing
- `xsl:choose` — like a switch statement; `xsl:when` clauses are tested top-to-bottom; `xsl:otherwise` is the default
- `xsl:sort` — sorts the node-set before iteration; `order="descending"` also available; `data-type="number"` for numeric sort
- `xsl:output method="html"` — also accepts `"xml"` and `"text"`; controls serialization of the result

OUTPUT METHODS

METHOD=	OUTPUT
"html"	HTML; void elements like <code>
</code> not closed
"xml"	XML; all elements properly closed
"text"	Plain text; all markup stripped

XSLT ELEMENT QUICK REFERENCE

ELEMENT	KEY ATTRIBUTE(S)	PURPOSE
<code>xsl:stylesheet</code>	version, xmlns:xsl	Root of every stylesheet
<code>xsl:output</code>	method, encoding, indent	Output serialization
<code>xsl:template</code>	match, name, mode	Define a transformation rule
<code>xsl:apply-templates</code>	select, mode	Delegate to matching templates
<code>xsl:call-template</code>	name	Call a named template
<code>xsl:value-of</code>	select	Output string value of XPath
<code>xsl:copy-of</code>	select	Copy nodes verbatim
<code>xsl:for-each</code>	select	Iterate over a node-set
<code>xsl:sort</code>	select, order, data-type	Sort within for-each or apply-templates
<code>xsl:if</code>	test	Conditional — no else branch
<code>xsl:choose</code>	—	Multi-branch conditional
<code>xsl:variable</code>	name, select	Declare an immutable variable
<code>xsl:param</code>	name, select	Template parameter

COMMON MISTAKES

- › Using `>` or `<` raw inside `test="..."` — XPath operators inside XML attributes must be escaped as `>` / `<`
- › Forgetting that `xsl:variable` is immutable — you cannot update a variable inside a loop; use recursion or `xsl:for-each` accumulation patterns instead
- › Omitting `xsl:apply-templates` in a template — without it, child nodes are silently dropped from the output

TIPS

- › Prefer template matching (`xsl:apply-templates`) over `xsl:for-each` — it produces more modular, reusable stylesheets
- › Use `xsl:copy-of` instead of `xsl:value-of` when you need to copy a subtree with its markup intact
- › Test stylesheets with `xsltproc input.xml transform.xml` on the command line for fast iteration

SUMMARY

- › `xsl:template match="..."` — rule triggered when a node matches the XPath pattern
- › `xsl:apply-templates` — recurse into child nodes; drives the transformation engine
- › `xsl:value-of` — output text; `xsl:copy-of` — output node tree
- › `xsl:if` (no else) · `xsl:choose/when/otherwise` (multi-branch)

XSLT STYLESHEET — XML TO HTML

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
  version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <!-- Output format ----- -->
  <xsl:output method="html" encoding="UTF-8" indent="yes"/>

  <!-- Root template - matches document root ----- -->
  <xsl:template match="/">
    <html><body>
      <h1>Book Catalog</h1>
      <xsl:apply-templates select="catalog/book"/>
    </body></html>
  </xsl:template>

  <!-- Template for each book ----- -->
  <xsl:template match="book">
    <div>
      <h2><xsl:value-of select="title"/></h2>
      <p>Author: <xsl:value-of select="author"/></p>

      <!-- xsl:if - single condition -->
      <xsl:if test="price > 10">
        <p>Price: <xsl:value-of select="price"/></p>
      </xsl:if>

      <!-- xsl:choose - multiple conditions -->
      <xsl:choose>
        <xsl:when test="@lang = 'en'"><span>English</span></xsl:when>
        <xsl:when test="@lang = 'fr'"><span>French</span></xsl:when>
        <xsl:otherwise><span>Other</span></xsl:otherwise>
      </xsl:choose>
    </div>
  </xsl:template>

  <!-- for-each with sort ----- -->
  <xsl:template match="catalog">
    <xsl:for-each select="book">
      <xsl:sort select="title" order="ascending"/>
      <p><xsl:value-of select="title"/></p>
    </xsl:for-each>
  </xsl:template>

</xsl:stylesheet>
```

PARSING MODELS

Three major models exist for parsing XML programmatically. **DOM (Document Object Model)** loads the entire document into a tree in memory — easy to navigate and modify, but memory-intensive for large files. **SAX (Simple API for XML)** streams the document and fires events (`startElement`, `endElement`, `characters`) as it reads — low memory, but read-only and forward-only. **StAX (Streaming API for XML)** is a pull-based streaming model where your code drives parsing iteration — a middle ground that is easier to use than SAX while remaining memory-efficient.

MODEL COMPARISON

- › **DOM** — full tree in RAM; random access; read/write; slow on large files
- › **SAX** — event-driven; push model; very low memory; read-only; forward-only
- › **StAX** — pull streaming; low memory; more control than SAX; Java-native
- › Browser JS uses DOM API natively via `DOMParser`
- › Python: `xml.etree.ElementTree` = lightweight DOM; `xml.sax` = SAX
- › Always check for parse errors — malformed XML throws, not silently fails

PARSING IN PYTHON, JS & JAVA

```
# — Python: xml.etree.ElementTree (DOM-like) —————
import xml.etree.ElementTree as ET

tree = ET.parse('catalog.xml')
root = tree.getroot()          # <catalog> element

for book in root.findall('book'):
    title = book.find('title').text # text content
    bid   = book.get('id')          # attribute value
    price = float(book.find('price').text)
    print(bid, title, price)

# XPath-like selection (limited subset)
books = root.findall('.//book[@lang="en"]')

# Build and write XML
new_book = ET.SubElement(root, 'book', id='bk103')
ET.SubElement(new_book, 'title').text = 'New Title'
tree.write('out.xml', encoding='UTF-8', xml_declaration=True)
```

NOTES

- `ET.parse()` — loads file into memory; use `ET.fromstring(xml_str)` to parse a string directly
- `.findall('book')` — finds direct children named `book`; `'.//book'` finds at any depth
- `.find('title').text` — the text content of an element; returns `None` if element is missing — guard against this
- `.get('id')` — attribute value; returns `None` if attribute absent (use `.get('id', 'default')` for a fallback)
- `ET.SubElement(parent, tag)` — creates and appends a new child element in one call
- `xml_declaration=True` — writes `<?xml version='1.0' encoding='UTF-8'>` at the top of the output file

BROWSER / JS DOM API

METHOD / PROPERTY	DESCRIPTION
<code>new DOMParser()</code>	Create a parser instance
<code>.parseFromString(str, mime)</code>	Parse XML string; mime: "text/xml"
<code>doc.querySelector(css)</code>	CSS selector on XML document
<code>doc.querySelectorAll(css)</code>	NodeList of all matches
<code>el.getAttribute(name)</code>	Read an attribute value
<code>el.textContent</code>	All text inside element (incl. descendants)
<code>el.children</code>	HTMLCollection of element children
<code>new XMLSerializer()</code>	Serialize DOM back to XML string
<code>.serializeToString(node)</code>	Node → XML string
<code>doc.querySelector("parsererror")</code>	Check for parse errors

PYTHON ELEMENTTREE API

METHOD	DESCRIPTION
<code>ET.parse(file)</code>	Parse file → ElementTree object
<code>ET.fromstring(str)</code>	Parse string → root Element
<code>tree.getroot()</code>	Get root Element of the tree
<code>el.find(path)</code>	First match (XPath subset)
<code>el.findall(path)</code>	All matches (list)
<code>el.get(attr)</code>	Attribute value or None
<code>el.text</code>	Text content of element
<code>el.tag</code>	Element tag name
<code>el.attrib</code>	Dict of all attributes
<code>ET.SubElement(parent, tag)</code>	Create and append child
<code>tree.write(file)</code>	Serialize to file

COMMON MISTAKES

- › Not checking for `None` after `.find()` — if the element is missing, accessing `.text` on `None` throws an `AttributeError`
- › Using browser `DOMParser` without checking for `<parsererror>` — a malformed document still returns a document object, not an exception
- › Loading huge XML files with DOM parsers — use SAX or StAX for files over ~50 MB to avoid memory exhaustion

TIPS

- › Use Python's `lxml` library for full XPath 1.0 support and much faster parsing than the standard library's `xml.etree`
- › In the browser, `doc.querySelector()` with CSS selectors is often simpler than XPath for basic navigation
- › For namespaced XML in Python ElementTree, element names appear as `{uri}localname` — prefix with the URI in braces in your selectors

SUMMARY

- › DOM — full tree in memory; best for small/medium files with random access
- › SAX — event push; best for huge files; read-only forward-only
- › Python: `ET.parse()` / `ET.fromstring()` · JS: `new DOMParser().parseFromString()`
- › Always guard against missing elements (`None`) and parse errors

XML IN THE REAL WORLD

XML underpins many formats you encounter daily without necessarily thinking of them as "XML." **SVG** graphics, **RSS/Atom** feeds, **SOAP** web service messages, Office Open XML (.docx, .xlsx internals), Maven pom.xml, Android layouts and manifests, and Spring configuration files are all XML vocabularies. Understanding the shared XML foundation means you can read, edit, validate, and transform any of them with the same tools and mental model.

COMMON XML TOOLING

- › `xmllint` — validate, format, and XPath-query from the command line
- › `xsltproc` — apply XSLT stylesheets from the command line
- › VS Code + XML extension (Red Hat) — schema-aware editing and validation
- › Oxygen XML Editor — full IDE for XML, XSD, XSLT, and XPath
- › Python `lxml` — fast C-backed DOM, full XPath 1.0, XSLT 1.0
- › Java JAXB — bind XML to Java objects with annotations
- › Online: xmllint.com, freeformatter.com/xml-formatter

NOTES

- RSS 2.0 is plain XML with no namespace — each `<item>` is one post/episode; `<guid>` should be a permanent unique URL
- SOAP wraps every request in `<soap:Envelope>` + `<soap:Body>`; the actual operation is a child of Body in its own namespace
- SOAP responses follow the same envelope structure; faults go in `<soap:Fault>` inside Body
- Maven POM must declare `xmlns="http://maven.apache.org/POM/4.0.0"`; Maven validates against its embedded XSD
- SVG is XML: you can open any `.svg` file in a text editor, apply XPath, or transform it with XSLT
- Android `AndroidManifest.xml` and layout files use the `android:` namespace — attributes like `android:id` reference the SDK's namespace URI

XML VS JSON

FEATURE	XML	JSON
Syntax	Tags + attributes	Keys + values
Schema	XSD, DTD, Relax NG	JSON Schema
Comments	✓ <code><!-- --></code>	✗ not supported
Namespaces	✓ built-in	✗ convention only
Attributes	✓ first-class	✗ key-value only
Mixed content	✓ text + elements	✗ not applicable
Query language	XPath / XQuery	JSONPath / jq
Transform	XSLT	jq / custom code
Verbosity	High	Low
Best for	Documents, SOAP, config	REST APIs, data

COMMON MISTAKES

- › Editing `pom.xml` or Android XML files without namespace awareness — removing or changing `xmlns` declarations breaks tool processing
- › Treating RSS/Atom feeds as plain text — they're full XML; parse with an XML parser to handle entities and encoding correctly
- › Converting XML to JSON naively — attributes, mixed content, and repeated element names all need special mapping decisions before writing conversion code

TIPS

- › Run `xmllint --format file.xml` to canonicalize indentation before committing XML files to version control — cleaner diffs
- › Use `xmllint --noout --schema schema.xsd doc.xml` in CI pipelines to validate config XML on every build
- › When converting XML → JSON, document your mapping rules for attributes (prefix with `@`?) and repeated elements (always array?) before writing any code

REAL-WORLD XML SNIPPETS

```
<!-- — RSS 2.0 feed ————— -->
<rss version="2.0">
  <channel>
    <title>My Blog</title>
    <link>https://example.com</link>
    <description>Latest posts</description>
    <item>
      <title>Post Title</title>
      <link>https://example.com/post-1</link>
      <pubDate>Mon, 01 Jan 2025 00:00:00 GMT</pubDate>
      <guid>https://example.com/post-1</guid>
    </item>
  </channel>
</rss>

<!-- — SOAP 1.1 request envelope ————— -->
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <GetStockPrice xmlns="http://example.com/stock">
      <StockName>AAPL</StockName>
    </GetStockPrice>
  </soap:Body>
</soap:Envelope>

<!-- — Maven pom.xml (partial) ————— -->
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0</version>
  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-core</artifactId>
      <version>6.1.0</version>
    </dependency>
  </dependencies>
</project>
```

XMLLINT COMMAND REFERENCE

COMMAND	PURPOSE
<code>xmllint file.xml</code>	Check well-formedness
<code>xmllint --format file.xml</code>	Pretty-print / indent
<code>xmllint --schema s.xsd f.xml</code>	Validate against XSD
<code>xmllint --dtdvalid d.dtd f.xml</code>	Validate against DTD
<code>xmllint --xpath "//book" f.xml</code>	Run XPath expression
<code>xsltproc t.xsl input.xml</code>	Apply XSLT stylesheet
<code>xmllint --noout --valid f.xml</code>	Silent validation (exit code only)

SUMMARY

- › SVG, RSS, SOAP, pom.xml, Android layouts — all standard XML vocabularies
- › XML favors documents + namespaces + schema; JSON favors lightweight data APIs
- › `xmllint` validates, formats, and XPath-queries from the command line
- › Know the XML → JSON mapping conventions before writing conversion logic